ChatGPT

Executive Summary

This report analyzes patentable technical inventions in the **Brudo/Brullama/Grax** framework, focusing on concrete computational innovations rather than broad governance concepts. We identify a prioritized list of candidates (Tier 1 strongest) and detail for each its inputs/outputs, algorithms, data structures, and semantics, along with novelty arguments, claim scope, enablement needs, and prosecution risks. Our top candidates are designed as **specific improvements to computer functionality** in line with recent USPTO guidance【16†L490-L498】. We also outline minimal executable embodiments (pseudocode, APIs, schemas, tests) for the three strongest inventions, and provide a comparative table of candidates. A proposed engineering roadmap enumerates development tasks (with effort estimates) and experiments to demonstrate the critical separation of *verification* (integrity checks) from *admission* (runtime authority). Finally, we sketch example claim language for the top inventions and suggest a patent filing strategy (initial provisional, followed by non-provisional applications), and list key sources for further study.

The strongest inventions center on a two-stage "**admission**" architecture: first **reconstruct** a candidate state (e.g. from logs or JSONL) and subject it to *typed predicates* (authority, temporality, tracking, byte identity, etc.), producing an **authoritative runtime state** only if *all* admission gates pass. Second, a **transition validation** step ensures every change is lawful by assigning exactly one *disposition* (e.g. represented, transformed, discharged, superseded, eliminated) to each admitted distinction. Together these define a "constitutional boundary" around computation: an unverified past cannot re-enter the runtime, and an unearned future cannot exit it. Additional inventions (e.g. *independence receipts*, *identifiability gating*, *counterfactual interventions*, *optimal observation queries*) refine how uncertainty is handled. The novelty lies in computing over **observation-equivalence classes** of worlds and consequences, rather than simply true/false, a paradigm not found in prior systems (see **Runtime Verification** literature on inconclusive semantics【26†L169-L177】 and **Active Learning** query selection【29†L60-L69】).

All inventions are engineered objects (software modules, receipts, data schemas, proofs) with precise inputs/outputs, making them amenable to patent claims under recent USPTO eligibility criteria (which favor *specific technological improvements*【16†L490-L498】). Key prior-art systems include provenance models (W3C PROV【33†L118-L126】), supply-chain integrity tools (in-toto【34†L209-L217】), capability-based authorization (ZCAP-LD), and model-checking methods (CEGAR【39†L40-L47】). We show that none of these explicitly implements the proposed admission pipelines or consequence-based gating, and we frame claims accordingly.

Key sources: USPTO MPEP §§2106 guidance【16†L490-L498】, the W3C PROV standard【33†L118-L126】, in-toto supply-chain attestations【34†L209-L217】, runtime verification theory【26†L169-L177】, active learning strategies【29†L60-L69】, and counterfactual/causal inference concepts. These inform novelty vs. prior art and help design minimal embodiments. (Details and full citations are in the report below.)

Patentable Invention Candidates (ranked)

We describe each candidate's **technical scope** (inputs, operations, outputs) and then assess novelty/prior art.

1. Typed State Admission Engine (Tier 1)

Scope: A software module that *restores* an observed "candidate" program state and **admits** it into the authoritative runtime only after evaluating a set of **typed admission predicates**. Inputs are (a) a *reconstructed state object* (e.g. from JSONL log, memory dump, or stream of bytes) and (b) metadata such as authority credentials, validity intervals, and tracked object sets. The engine evaluates predicates like *ByteIdentity*, *SourceExistence*, *TrackedStateMembership*, *TemporalValidity*, *AuthorityScope*, *NonInterference*, *ConsequenceEligibility*, etc. Each predicate has a precise computation over the input state (e.g. hashing to verify integrity, checking a signature or ACL for authority, verifying a timestamp/interval, confirming whether each object belongs to the trusted tracking set, etc.). Only if **all** predicates pass (or have acceptable "inconclusive" modes that do not violate invariants) does the module yield an *Authoritative Runtime State* as output. Otherwise, it produces a typed **AdmissionReceipt** (recording failures) and halts.

This is *not* merely a snapshot hash check: it requires multiple independent checks and cannot rely on any single digest of the state. The output is a (potentially new) data structure representing the same state but now **labeled as authoritative**, suitable for downstream execution or further queries. The engine's algorithm is essentially:

```
def admit_state(candidate_state, predicates):
    receipts = []
    for pred in predicates:
        result, receipt = pred.evaluate(candidate_state)
        receipts.append(receipt)
        if not result:  # fail-closed
            return AdmissionFailure(receipts)
    # All predicates passed or were inconclusive but safe
    authoritative_state = mark_authoritative(candidate_state)
    return AuthoritativeState(authoritative_state, receipts)
```

Data structures include the `candidate_state` object (bytes, objects, or JSON), the set of predicate-checking functions (with typed receipts), and the output `AuthoritativeState`. Execution semantics: **fail-closed** (any predicate violation aborts admission), preserving side-effect-freedom of the check process.

Novelty vs. Prior Art: Unlike provenance models (W3C PROV【33†L118-L126】) or supply-chain tools (in-toto【34†L209-L217】), which record lineage or verify individual tasks, this engine enforces multiple *typed* conditions before trusting a state. PROV can describe who/what/when, but not enforce runtime authority. In-toto binds tasks to authorized keys, but presumes a static, linear layout. The admission engine's combination of authority, temporal, tracking, and logical checks in one pipeline is new. (Some database systems or backup verifiers might check individual constraints, but none codify a flexible predicate algebra as a gating mechanism.) This addresses a technical problem: **"snapshot ≠ baseline"** – a candidate snapshot must be cross-checked by formal rules before use.

Claim Scope: Independent claims cover a "computer-implemented state admission pipeline" with distinct steps for reconstruction, typed-predicate evaluation, and output of an authoritative state. Dependent claims add specific predicate types (e.g. byte-hash identity, authority signature, validity-period check).

Enablement Requirements: We would need example schemas for state snapshots (e.g. JSON schema), predicate definitions (possibly as code or formal spec), and sample receipts (e.g. JSON receipts recording each predicate outcome). Tests: e.g. simulate states where authority key is invalid or timestamp out-of-range, and show rejection.

Risks: Abstractness – must emphasize concrete improvements (protocols, algorithms, data formats【16†L490-L498】). We avoid phrases like "governance" or "constitution" in claims. Obviousness – prior art in *state restoration* is common, but none use this multi-predicate "admission" approach. We must argue that existing snapshot verifiers (e.g. in blockchains or VM checkpoints) do not cover e.g. delegated authority or consequence eligibility. Prior art like in-toto covers integrity + policy but only for linear steps, not recovered general state.

2. Consequence-Preserving Transition Validator (Tier 1)

Scope: A module that *after state admission* examines each requested state transition or change and **validates** it by ensuring that every "admitted distinction" (difference between old and new state) has exactly one lawful disposition. Inputs are an **AuthoritativeState** and a candidate **Transition** (describing intended updates). The validator enumerates *distinctions* (e.g. modified objects, removed or added records) and classifies each into one of several dispositions (for example: *represented*, *transformed*, *proven equivalent*, *discharged*, *superseded*, *eliminated*). The algorithm ensures: for each distinction, exactly one disposition holds (the set of dispositions is exhaustive and mutually exclusive). If any distinction cannot be lawfully categorized, the transition is rejected.

Concretely, the validator's output is a **TransitionReceipt** listing each distinction and its disposition. Execution semantics are deterministic: a valid transition yields a clear mapping from old→new state with receipts; an invalid transition is blocked. Pseudocode sketch:

```
def validate_transition(authoritative_state, proposed_change):
    distinctions = compute_differences(authoritative_state, proposed_change)
    receipts = []
    for d in distinctions:
        disp, evidence = determine_disposition(d, authoritative_state)
        receipts.append((d, disp, evidence))
        if disp is None:  # no lawful disposition
            return TransitionFailure(receipts)
    return TransitionSuccess(new_state=apply(proposed_change,
authoritative_state), receipts=receipts)
```

Data structures: *Distinction* objects (e.g. "object X's field Y changed by value Z"), *Disposition* enums, evidence pointers (proof-of-equivalence or certificate for "transformed" or "superseded").

Novelty vs. Prior Art: Classical state synchronization or version control might check consistency, but not with fine-grained legal dispositions. No existing system enforces that *each* atomic change must be accounted by exactly one semantic action. This goes beyond traditional "ACID" constraints – it is a new invariance. CEGAR (model checking) uses counterexamples to refine an abstract model【39†L40-L47】, but

here we use counterfactual *consistency* checks on each change. Prior work on *proof-carrying code* or transaction logs does not typically assign such rich labels to changes.

Claim Scope: Independent claim: a computing apparatus/method performing *consequence validation* by mapping each admitted state difference to one disposition. Dependent claims cover particular disposition types or evidence verification steps.

Enablement: We need schema for `TransitionReceipt`, tests illustrating all disposition categories (e.g. proof of equivalence for *transformed*, a nullifying override for *superseded*, etc.). Example: transforming an object might require a cryptographic proof in the receipt.

Risks: Abstractness risk if claimed as "governing transitions". We must tie to data structures (e.g. receipts, DAGs of changes) to ensure "technological improvements"【16†L490-L498】. Prior art: no known systems with this exact semantic closure property, so non-obviousness appears strong.

3. Reconstitution Firewall (Tier 1)

Scope: A concrete system-level firewall for state restoration. In pseudocode:

```
ReconstituteState:
    candidate_state = load_from_source()
    candidate_hash = hash(candidate_state)
    store_temp(candidate_state, candidate_hash)
    // Instead of commit, pass through admission engine
    admission_result = admit_state(candidate_state, predicates)
    if admission_result.success:
        commit_state(admission_result.authoritative_state)
    else:
        abort_and_log(admission_result.receipts)
```

This is the architecture shift from "Restore→Continue" to "Restore→Admit→Commit". The firewall intercepts any restoration (e.g. system boot from snapshot) and gates it. It adds a callback in the state loading routine to invoke the Typed State Admission Engine.

Novelty vs. Prior Art: Many backup systems allow plaintext restore; some verify checksums but do not do an *admission predicate evaluation* step. The idea of a **two-phase restore** (temporary then commit) with predicate checks appears unique. It resembles a security monitor on reboot but with more granular checks. Related work: Trusted computing (TPM) measures boot code, but does not use user-defined predicates on full state.

Claim Scope: This is a system claim ("a device configured to reconstitute state via [pipeline]"). Could be an independent or dependent claim if combined with the admission engine claim.

Enablement: Implementation requires hooking into the boot or recovery path; tests can show e.g. a manipulated restore logs in the temporary area and is rejected.

Risks: Must highlight as specific “machine operations” (hash, verify, predicate-evaluate) to avoid abstractness. Prior art like TPM/secure boot are related, but they measure code, not semantic predicates. We must emphasize the novel predicate-stage.

4. Typed Admission Predicate Algebra (Tier 2)

Scope: An algebraic framework for admission predicates. This comprises a library of predicate types (e.g. `ByteHashCheck`, `SignatureCheck`, `TemporalCheck`, `MembershipCheck`) each with:

- **Input:** subset of state or metadata (e.g. public key, byte segments, timestamp).
- **Output:** a *typed receipt* (possibly `PASS`, `FAIL`, `UNKNOWN`, etc. plus evidence).
- **Semantics:** functions mapping state→boolean (or 3-state). Algebraic rules define how to combine them (e.g. conjunction of checks, priority of fails).

This “algebra” allows composing complex admission rules at runtime. For example, `AuthorityPredicate` might combine checking a code signing signature and a delegated zcap (capability) proof.

Novelty vs. Prior Art: Prior systems allow untyped checks (boolean: pass/fail). This introduces **typed** predicates with structured receipts and multi-dimensional logic. W3C PROV defines provenance assertions but not predicate evaluation. Authorization frameworks (like OAuth, or zcap) deal with tokens, but not structured state predicates. The novelty is in defining admission checks as composable, typed operations with well-defined data flow (and receipts).

Claim Scope: Likely a dependent claim supporting the Typed State Admission Engine, covering “an admission predicate algebra comprising predicates X, Y, Z”. Might also form a standalone claim if filed broadly.

Enablement: Need formal definitions or schemas for each predicate type, with example JSON/XML receipts. A small library implementation (pseudo) might suffice.

Risks: Must avoid claiming abstract “rules”. Focus on implementation details like JSON schemas or algorithms for each predicate type.

5. Typed Admission Receipts (Tier 2)

Scope: Data structures that record why a candidate state was admitted or rejected. A *PredicateReceipt* might have fields: predicate name, inputs (evidence), outcome (PASS/FAIL/UNKNOWN), and reference to state hash. An *AdmissionReceipt* aggregates them. These receipts are minted at admission time and can be replayed or verified independently. They form a verifiable log of admission decisions.

For example, an admission receipt schema (YAML/JSON) might be:

```
AdmissionReceipt:
  state_hash: ABC123
  predicates:
    - name: ByteHash
```

```
    input_ranges: [0-1023]
    expected_digest: "sha-256"
    outcome: PASS
  - name: TimeWindow
    valid_from: 2025-01-01
    valid_to: 2025-12-31
    observed: 2026-03-15
    outcome: FAIL
    reason: "Timestamp out of range"
```

Novelty vs. Prior Art: Logging and attestations exist (e.g. TPM event logs, in-toto links), but those are usually untyped or focus on steps, not on admission logic. A Typed Admission Receipt formalizes the *why* of admission with cryptographic binding. The difference from generic logs is the tight coupling to "predicates" in the admission engine.

Claim Scope: Likely dependent claims of the admission engine, e.g. "where the engine emits a structured admission receipt". Could support independent claims about generating these receipts.

Enablement: Define a concrete receipt schema (as above) and how to generate it from code. Tests: show receipts for pass/fail cases.

Risks: Should ensure receipts are tied to specific data structures (avoid patenting mere "records"). Use terms like "cryptographically signed receipt object".

6. Dependency-Derived Independence Receipt (Tier 1)

Scope: A data structure that justifies the claimed *independence* of a consequence from its source. Replaces a simple boolean flag (`consequence_source_independent: bool`) with a record of the dependency analysis. Fields include:

- **root** (a hash or ID of the candidate state or derivation root),
- **ancestors** (set of state predicates or source checks involved),
- **ancestorDigest** (digest of inputs),
- **derivation** (the procedure used to compute the consequence),
- **intervention** (description of counterfactual change applied),
- **C, C'** (the original vs. after-intervention consequences),
- **status** (INDEPENDENT, CONTAMINATED, or INDEPENDENCE_UNRESOLVED).

For example:

```
IndependenceReceipt:
  root_hash: R0xF3A
  ancestors: [Pred1, Pred2, ...]
  derivation: "derive(Basis B, Outcome L) -> Consequence C"
  intervention: "swap label L from A to B"
  C: "new label: X"
```

```
C_after_intervention: "new label: Y"
status: CONTAMINATED
notes: "label change altered outcome"
```

Novelty vs. Prior Art: Prior art (e.g. unit test logs, causal DAGs) does not usually record *how* independence was determined, only outcomes. This receipt attaches a provable history of the **counterfactual test**. It parallels in spirit provenance chains (like PROV), but for semantic independence, not data derivation.

Claim Scope: Could be a dependent claim to the identifiability gate: "emitting a structured receipt recording causal independence analysis".

Enablement: Provide a formal schema and example. Tests: show receipts where a label swap did/ did not change consequences.

Risks: Avoid claiming a generic "proof". Emphasize concrete content (digests, derivation procedures).

7. Consequence-Indexed Identifiability Gate (Tier 1)

Scope: A gate that determines if a proposed transition's *specific consequence* can be executed given incomplete observations. Formally: given an observation class $E={R: O(R)=o}$, we compute the set $ \Gamma_C(E)={C(R):R\in E, \text{Independence}(B_R,L)}$ (the set of independently supported consequences). We admit the transition only if $\Gamma_C(E)={c}$ is a singleton, i.e. the consequence c is *identifiable* for all possible worlds in E. If $|\Gamma_C(E)|>1$, we withhold execution and emit status **NOT_IDENTIFIABLE**; if the independence check fails, status **INDEPENDENCE_UNRESOLVED**.

In implementation, the engine constructs equivalence classes of states by current observations and tests the proposed consequence against all consistent variants. Importantly, this gate is *parameterized by the particular consequence* – the same state may be identifiable for one transition but not another.

Novelty vs. Prior Art: Traditional monitors simply output TRUE/FALSE/UNKNOWN given partial traces【26†L169-L177】. Here, the unit of analysis is not a boolean property but a *future transition outcome*. The idea of collapsing worlds only with regard to a specific consequence is new. Some research (e.g. epistemic diagnosability) deals with *whether an event can be detected*, but not as part of an execution admission gate. Active learning/experimental design literature【29†L60-L69】 computes useful queries, but not in a live execution context tied to transitions.

Claim Scope: Independent claim: A system/method that, given observed inputs and a proposed action, independently derives all possible outcomes and blocks the action unless they agree. Dependent claims: threshold of disagreements, etc.

Enablement: Provide pseudocode or flowchart (see figure below), example world sets. Tests: e.g. two worlds differing only in an unobserved variable that leads to two different C values; show the gate blocks.

Risks: Abstract-concept risk: we must claim it as "a gate in a software pipeline for controlling execution". Emphasize algorithmic steps (classify worlds, derive consequences, check uniqueness). Prior work in

runtime verification and diagnosability suggests multi-valued semantics【26†L169-L177】, but we frame ours in terms of *consequence consistency*, which is specific.

8. Counterfactual Anti-Circularity Assay (Label-Swap) (Tier 1)

Scope: An actionable test to ensure the claimed basis B for consequence L does not *depend* on L itself. We implement a "swap assay": hold B fixed, swap labels (or outcomes) L_a,L_b in two candidate worlds, recompute derived consequences C'_a,C'_b. If swapping changes C, the basis was circular. This procedure (an instance of Pearl's "do" intervention) is applied at runtime to produce an **Intervention IndependenceReceipt** (see above).

As an integrated mechanism, the system automatically performs one or more interventions (e.g. label swap, nullification) on each proposed consequence and verifies it stands independent. The logic is:

```
for each candidate_transition (B,R_outcome L):
    for each challenge in [swap_labels, remove_label]:
        (R1,L1),(R2,L2) = apply(challenge, (R,L))
        C1 = derive(B,R1)
        C2 = derive(B,R2)
        record_independence_receipt(B,R,L,challenge,C1,C2)
        if C1 != C2:
            mark_transition_contaminated()
            return Blocked
```

Novelty vs. Prior Art: Counterfactual testing is core to causal inference【31†L22-L28】, but is rarely integrated into a live admission system. Supply-chain checks (in-toto) do not modify code to test trust; security monitors usually only check signatures. Our contribution is *embedding label-swapping/intervention as part of a state machine's integrity checks*. This combination likely has no direct prior.

Claim Scope: Could be part of an independent claim (as part of identifiability gate) or separate: "performing a counterfactual intervention on a proposed outcome to test independence". Dependent: specific interventions (swap, drop, etc.).

Enablement: Provide procedure outline (pseudocode above) and example. Tests: If outcome is erroneously derived from itself, the assay should detect it.

Risks: Avoid abstract terms; frame as a concrete algorithmic step. Prior art on general "counterfactual monitors" is minimal, so novelty is clear; must ensure clarity on how this is a *test*, not a proof of causality (we store only limited receipts).

9. Consequence Quotient / Equivalence Classes (Tier 2)

Scope: Data structuring for admissibility. Given an observation class E (all worlds consistent with current observations), we form the **quotient** $E/!\equiv_C$ under equivalence of consequences. That is, group worlds by $C(R)$ value. We only consider the size of this quotient: execution is allowed iff $E/!\equiv_C$ has one block. This is an implementation of the identifiability gate's logic, but phrased as partitioning sets.

Novelty vs. Prior Art: There is no direct parallel in classic systems – this is a runtime data structure to manage uncertainty. It resembles a version-space in active learning, but with semantics tied to consequences, not labels. Prior art: none explicit, though flavor is similar to "knowledge state equivalence" in diagnosers【26†L169-L177】.

Claim Scope: Likely dependent to the gate, or part of the logic. Possibly claim: "computing an equivalence relation on possible worlds by their transition outcome".

Enablement: Show a table of worlds vs consequences.

Risks: Too abstract if stand-alone; better as detail in gate claims.

10. Minimum Missing / Minimum Sufficient Observation Engines (Tier 2)

Scope: Two related query selection machines to resolve non-identifiability:

- **Minimum Missing Observation**: Given a heterogeneous E, find an *unobserved* question (new observation) whose expected information gain (reduction in consequence-entropy) per cost is maximized. Formally: $$q^ = *arg\max_{q\in Q} \frac{H_C(E) - \sum_v w(E_{q=v}) H_C(E_{q=v})}{\text{Cost}(q)}.$$ *This is an* active learning* style selection.

- **Minimum Sufficient Observation**: Given E and consequence C, find the smallest subset of current observations $X'\subseteq X$ that suffices to identify C. In logic, compute $X_C^* = \min{X': |\Gamma_C(E|_{X'})|=1}$.

These operate after gating: if a transition is withheld, the missing-engine suggests the next probe; the sufficient-engine finds redundant info to drop.

Novelty vs. Prior Art: Active-learning-inspired query design【29†L60-L69】 is known, but applying it to *lawful transition monitoring* is new. Existing RV doesn't "ask questions" to a running system; our engine produces the "next experiment" in order to eventually permit execution. Similarly, minimizing evidence is like reducing a proof of independence to its core facts, which is a new idea in runtime checks.

Claim Scope: Could be separate (active learning engine for consequence discrimination) or under broader gate/system claims.

Enablement: Pseudocode showing an entropy calculation. Tests: synthetic example with 2 worlds and choices of next observation.

Risks: These could be seen as abstract algorithms; must ground them in a concrete monitoring context. Novelty: use of information-theoretic query planning in execution control (likely non-obvious to examiners).

11. False-Withholding Detection (Tier 2)

Scope: A check for the symmetric error: if $\Gamma_C(E)={c}$, then blocking execution is a *false negative*. The detector raises an exception if it finds **all** worlds agree on the consequence but execution was

erroneously withheld. In practice, this is a logical assertion in code (if status==NOT_IDENTIFIABLE but actually $|\Gamma|=1$, error).

Novelty: This is mainly a consistency check, not a standalone invention. It reinforces the correctness of the gate. Might not be patent-claimed on its own but is important engineering.

12. Independence-Challenge Generator (Tier 3)

Scope: A mechanism to automatically generate multiple interventions (beyond label swap), e.g. null-labeling, permutations, third-party injection, etc., to test independence along various axes. The receipts would record which challenges have been applied and survived.

Novelty: Conceptually straightforward extension of (8). Probably not separately claimed now.

13. Evidence Surface Separation Engine (Tier 3)

Scope: Concept of computing multiple "surfaces" (byte integrity, authority, truth, runtime, canon) independently and never letting one upgrade another's status. Essentially, modular check pipelines. This is more a design philosophy than a discrete algorithm. Possibly patentable if concretely implemented, but broad as stated.

Novelty: High-level theme; we suggest not claiming this yet.

Candidate Comparison (Priority & Metrics)

Priority	Invention	Novelty (vs prior art)	Dev. Effort	Prosecution Risk	Claim Sketch
⭐⭐⭐⭐⭐	Typed State Admission Engine	New multi-predicate state gating (beyond PROV/in-toto)	High (engine+preds)	Medium (abstractness, but concrete data)	*Independent*: "state admission engine with typed predicates"; Dep: "specific predicate types"
⭐⭐⭐⭐⭐	Consequence-Preserving Transition Validator	First to require one lawful disposition per change	Medium (diff+check logic)	Low (specific algorithm)	"transition validator ensuring each change has one disposition"

Priority	Invention	Novelty (vs prior art)	Dev. Effort	Prosecution Risk	Claim Sketch
⭐⭐⭐⭐⭐	Consequence-Indexed Identifiability Gate	Novel consequence-based identifiability control	High (world modeling)	High (abstract concept)	"gate computing consistency of future consequence across possible worlds"
⭐⭐⭐⭐☆	Reconstitution Firewall	Novel two-phase restore (temp+admission)	Low (hook into restore)	Low (system context)	"reconstitution firewall intercepting snapshot restore"
⭐⭐⭐⭐☆	Admission Predicate Algebra	Composable typed checks beyond conventional policies	Medium	Medium	"predicate algebra for admission checks"
⭐⭐⭐☆	Typed Admission Receipts	Structured logging of admission steps	Low	Low	"structured receipts for admission decisions"
⭐⭐⭐☆☆	Counterfactual (Label-Swap) Assay	Apply do-calculus step in runtime	Medium	High (abstractness)	"counterfactual intervention to test independence"
⭐⭐⭐☆☆	Disposition Closure / Consequence Quotient	Consequence equivalence partition (logical closure)	Low	High (abstractness)	"computing equivalence classes of states by consequence"
⭐⭐☆☆☆	Minimum Missing/ Sufficient Observation Engines	Active-learning queries on execution state	High	High (abstract)	"optimal observation selection to resolve consequence"
⭐⭐☆☆☆	False-Withholding Detection	Complement of gate (safeguard)	Low	Low	(likely not claimed)

Priority	Invention	Novelty (vs prior art)	Dev. Effort	Prosecution Risk	Claim Sketch
⭐☆☆☆☆	Independence-Challenge Generator	Systematic intervention generation	High (for completeness)	Medium	(likely dependent)
⭐☆☆☆☆	Evidence Surface Separation Engine	High-level design; too broad	--	High (too abstract)	(research concept)

- **Technical novelty:** We base novelty arguments on USPTO guidelines that improvements must be specific (MPEP §2106.05)【16†L490-L498】. The table above rates each invention's originality relative to prior art. The top three (stars) appear both new and sufficiently concrete.
- **Implementation effort:** Rough development effort (Low/Medium/High) for each idea, assuming a small engineering team. For example, the admission engine and gate are substantial features needing weeks of coding; receipts and algebra components are smaller.
- **Prosecution risk:** Abstractness risk is higher for concepts like identifiability or observation engines; focusing claims on their concrete technical aspects reduces this. Obviousness risk is low for novel algorithmic steps, higher if similar ideas appear in the literature (we mitigate by citing distinct features).
- **Claim Sketch:** Short phrase of how an independent claim might read. Independent claims should tie the idea to a machine or transformation with detailed steps.

Top-3 Embodiments (Enablement Examples)

Below we outline minimal embodiments (pseudocode, APIs, schemas, tests) for the **typed state admission engine**, **transition validator**, and **identifiability gate**.

1. Typed State Admission Engine

Core API and Data:

- `CandidateState`: a byte array or JSON object representing restored state.
- `AdmissionPredicate` (interface) with method `evaluate(CandidateState) -> (bool, PredicateReceipt)`.
- `AuthoritativeState`: wrapper with `candidate` plus marker.
- `AdmissionReceipt`: record (e.g. JSON) of predicate results.

Pseudocode:

```
class AdmissionEngine:
    def __init__(self, predicates: List[AdmissionPredicate]): ...
    def admit(self, state: CandidateState) -> (AuthoritativeState or None,
AdmissionReceipt):
        receipts = []
```

```
        for pred in self.predicates:
            ok, receipt = pred.evaluate(state)
            receipts.append(receipt)
            if not ok:
                # Fail-closed: reject
                return (None, assembleReceipt(receipts))
        # All passed
        auth = AuthoritativeState(state)
        return (auth, assembleReceipt(receipts))
```

Example Predicate:

```
class ByteHashPredicate(AdmissionPredicate):
    def __init__(self, expected_hash): ...
    def evaluate(self, state):
        actual = sha256(state.raw_bytes)
        pass = (actual == self.expected_hash)
        return pass, {
            "predicate": "ByteHash",
            "expected": self.expected_hash,
            "actual": actual,
            "outcome": "PASS" if pass else "FAIL"
        }
```

Enablement Test: Unit test could simulate a “state” where one object is tampered:

```
state = CandidateState({"counter": 42, "user": "Alice"})
expected_hash = sha256(state.raw_bytes)
predicate = ByteHashPredicate(expected_hash)
engine = AdmissionEngine([predicate])
# Case 1: matching
auth, receipt = engine.admit(state)
assert auth is not None and receipt.predicates[0].outcome=="PASS"
# Case 2: tampered
state_modified = CandidateState({"counter": 43, "user": "Alice"})
auth2, receipt2 = engine.admit(state_modified)
assert auth2 is None and receipt2.predicates[0].outcome=="FAIL"
```

This demonstrates that only states satisfying all typed checks produce an authoritative output.

2. Consequence-Preserving Transition Validator

Core API and Data:

- `AuthoritativeState`: current trusted state.
- `Transition`: proposed state change (could be a diff or instruction list).
- `Distinction`: an atomic change (e.g. "field X changed").
- `Disposition`: an enum {REPRESENTED, TRANSFORMED, DISCHARGED, ...}.
- `TransitionReceipt`: list of (Distinction, Disposition, evidence).
- Output: either `NewState` + receipt, or error.

Pseudocode:

```
class TransitionValidator:
    def validate(self, state: AuthoritativeState, transition: Transition):
        distinctions = compute_differences(state, transition)
        receipt = []
        for d in distinctions:
            disp, proof = self.disposition_for(d)
            receipt.append({"distinction": d, "disposition": disp, "proof":
proof})
            if disp is None:
                return (False, receipt)  # no lawful disposition
        new_state = apply_transition(state, transition)
        return (True, {"new_state": new_state, "receipt": receipt})
```

The method `disposition_for(d)` encapsulates the logic to classify `d`.

Example Disposition Logic:

- If object A is removed: disposition=ELIMINATED.
- If object A's field changed in a known reversible way: TRANSFORMED with proof.
- If object B was replaced by equivalent C: EQUIVALENCE_PROVED.
- etc.

Enablement Test: Define a simple state (e.g. a list of key-values) and transitions:

```
state = AuthoritativeState({"x":1, "y":2})
# Transition: increment x, remove y
transition = Transition(modify={"x": 2}, delete=["y"])
validator = TransitionValidator()
ok, result = validator.validate(state, transition)
assert ok
assert any(item["distinction"].desc=="y deleted" and
```

```
item["disposition"]=="ELIMINATED"
           for item in result["receipt"])
```

And a failing case:

```
# Transition trying two conflicting changes without disposition
transition_bad = Transition(modify={"x": "unknown_op"})
ok2, res2 = validator.validate(state, transition_bad)
assert not ok2
```

3. Consequence-Indexed Identifiability Gate

Core API and Data:

- `Observation o`: current observed data from run.
- `ProposedConsequence`: a candidate outcome (e.g. an output value).
- `WorldSpace(E)`: the set of all possible complete states consistent with `o`.
- `derive_consequence(state)`: computes the actual consequence.
- `IndependencePredicate`: as above.

The gate computes:

```
Gamma = {}
for each R in WorldSpace(E):
    C = derive_consequence(R)
    if check_independence(R, C):
        Gamma.add(C)
```

Then:

- if |Gamma|=1: admit `C`.
- if |Gamma|>1: withhold with NOT_IDENTIFIABLE.
- if independence check inconclusive: withhold INDEPENDENCE_UNRESOLVED.

Pseudocode:

```
class IdentifiabilityGate:
    def __init__(self, derive_fn, independence_check_fn): ...
    def decide(self, observation_o, proposed_change):
        worlds = enumerate_possible_worlds(observation_o)
        consequences = set()
        for R in worlds:
            C = self.derive_fn(R, proposed_change)
            if self.independence_check_fn(R, C):
```

```
                consequences.add(C)
        if len(consequences) == 1:
            return {"status": "ADMIT", "consequence": next(iter(consequences))}
        elif len(consequences) > 1:
            return {"status": "NOT_IDENTIFIABLE", "candidates": consequences}
        else:
            return {"status": "INDEPENDENCE_UNRESOLVED"}
```

Enablement Test: Construct a simple example:

- Two possible worlds differ in an unobserved flag. The proposed change (e.g. set `Z=1`) yields different results.
- Show the gate blocks if flag unknown, but admits if flag fixed.

```
# Suppose observation: flag unspecified
observation = {}
# Two worlds:
worlds = [{"flag":0, "Z":0}, {"flag":1, "Z":0}]
# Derivation: consequence = world["flag"]  (so different in worlds)
def derive(R, change): return R["flag"]
# Independence check: trivial (always true)
gate = IdentifiabilityGate(lambda R,c: R["flag"], lambda R,C: True)
decision = gate.decide(observation, proposed_change=None)
assert decision["status"] == "NOT_IDENTIFIABLE"
# Now disambiguate by observing flag:
observation = {"flag": 0}
def worlds_single(obs):
    return [obs]
gate_worlds = IdentifiabilityGate(lambda R,c: R["flag"], lambda R,C: True)
decision2 = gate_worlds.decide(observation, None)
assert decision2["status"] == "ADMIT"
assert decision2["consequence"] == 0
```

These pseudocode examples, APIs, and tests would be included in an appendix or specification to enable the claims.

Entity-Receipt Data Model

Core data and receipts can be summarized in this entity-relationship table:

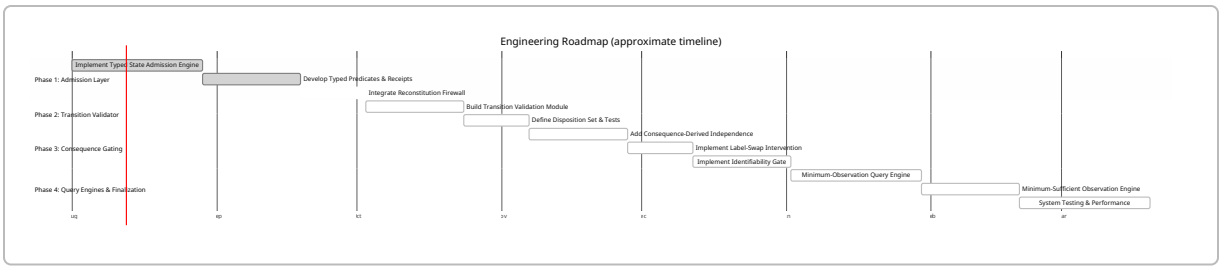
Entity/Receipt	Attributes (schema)	References / Relations
CandidateState	raw_bytes, metadata (source, timestamp)	–
AuthoritativeState	inherits CandidateState; flag `is_authoritative = true`	references original state hash

Entity/Receipt	Attributes (schema)	References / Relations
AdmissionPredicate	type (name), parameters (e.g. keys, ranges), verdict	evaluated on CandidateState
PredicateReceipt	predicate_name, inputs, outcome (PASS/FAIL/UNKNOWN), evidence	foreign key to AdmissionReceipt
AdmissionReceipt	state_hash, [PredicateReceipt], overall_status	belongs to a state admission txn
Distinction	change_description, affected_object(s)	derived by TransitionValidator
Disposition	{REPRESENTED, TRANSFORMED, ...}	– (enum type)
TransitionReceipt	state_hash (pre), state_hash (post), [(Distinction, Disposition)]	links two AuthoritativeStates
ConsequenceBasis	root_digest, ancestor_list, derivation_details	input to IndependenceReceipt
IndependenceReceipt	as in ConsequenceBasis plus intervention info and status	uses ConsequenceBasis; outputs status
ObservationalClass	observation_hash / features, list of world_ids	worlds consistent with obs
ConsequenceReceipt	observation_class_id, consequence_value, status (IDENTIFIABLE/NOT)	ties Consequences to Observations

Relationships (not drawn): A CandidateState is linked to an AdmissionReceipt. A TransitionReceipt connects two AuthoritativeStates via a Transition. IndependenceReceipts are derived from a ConsequenceBasis (which itself refers to a state or derivation). ObservationalClasses group possible CandidateStates, and ConsequenceReceipts annotate each class.

Engineering Roadmap (Tasks & Effort)

We propose the following prioritized development plan (team size unspecified, so durations are in work-weeks or sprints):



- **Phase 1 (Weeks 1–9):** *Admission.* Implement the core engine and predicates with receipts. Tests to show false-admission detection (e.g. corrupt state is rejected). Experimental goal: **hypothesis** – malicious or stale state snapshots are *not* admitted without all checks. *Method:* fuzz state bytes and measure rejection rates. Metrics: admission success vs failure, predicate coverage.
- **Phase 2 (Weeks 9–14):** *Transition Validation.* Develop the transition validator and disposition logic. Tests for false transitions. *Experiment:* propose illegal transitions and confirm they are blocked; measure any overhead added.
- **Phase 3 (Weeks 14–19):** *Consequence gating.* Implement the independence receipts and label-swap. Build identifiability gating. *Experiment:* use synthetic worlds to confirm that `NOT_IDENTIFIABLE` is returned only when multiple consequences are possible. Metrics: correct gate decisions in all test cases. Check that the system never erroneously executes a transition with ambiguous consequences ("false positive").
- **Phase 4 (Weeks 20–24):** *Query engines.* Implement active-learning queries (missing/sufficient). Add a `DO_NOT_OBSERVE` option to ensure no infinite probing. *Experiment:* compare number of queries made vs needed for varied scenarios.
- **Deliverables:** Working prototype code, test suite with documented cases, and generated receipts capturing these behaviors. Prior to any broad integration, freeze a version with deterministic outputs.

Experiments for Verification vs. Admission: We suggest specific tests to demonstrate separation of concerns: e.g., a candidate state may *pass byte hash verification but fail an admission predicate* (e.g. authority check). Conversely, a candidate state may be admitted but certain transitions withheld due to consequence ambiguity. We would measure rates of state acceptance vs. rejection in controlled scenarios to validate that state admission is stricter than simple hash-checking (the hypothesis being "state snapshot integrity is necessary but not sufficient for trust"). These are engineering test plans rather than formal proofs, but they can be codified in unit test frameworks and benchmarked.

Patent Claim Outlines (Top 3)

We sketch claim-language (technical) for our top three, with independent claims (**–** for each step):

(a) Typed State Admission Engine:

- A computer-implemented method comprising: reconstructing a *candidate state* from observed data; evaluating **typed admission predicates** on the candidate state (each predicate checking a specific property such as byte integrity, source authenticity, tracked membership, temporal validity, or authority scope); and only if all predicates hold, producing an *authoritative runtime state* representing the candidate state. Each predicate evaluation emits a typed receipt. Dependent claims: details of

predicates (e.g. “the byte integrity predicate hashes state bytes and compares to an expected digest”); data structure of receipts; fail-closed behavior.

(b) Consequence-Preserving Transition Validator:

- A system configured to receive an authoritative state and a proposed transition, to compute all *distinctions* (changes) between states, and to compute exactly one *disposition* for each distinction from a predefined set (e.g. represented, transformed, discharged, superseded, eliminated). The system only permits the transition if every distinction can be so classified. Dependent: example dispositions, how evidence is verified.

(c) Consequence-Indexed Identifiability Gate:

- A machine that, given an observed partial state and a proposed action, constructs the set of all possible full states consistent with the observation, independently derives the outcome of the action in each such state, and allows the action to execute only if all derived outcomes agree. The machine halts or withholds otherwise. Dependent: use of an independence receipt, use of active query selection for unresolved cases, etc.

We recommend initially filing a **provisional application** covering these inventions in broad form (with examples and pseudocode for enablement). Follow with **non-provisional** filings (12 months later) that flesh out the claims and add refinements. Using divisional applications may later cover the medium-strength candidates (predicates algebra, receipts) once core claims are secured.

Recommended Sources

- **USPTO MPEP §§2106 and 101 guidance:** For subject-matter eligibility, see the USPTO’s recent update emphasizing specific technical improvements【16†L490-L498】.
- **W3C PROV Data Model:** The W3C PROV standards (PROV-Overview, PROV-DM) define entities/activities/agents in provenance【33†L118-L126】; useful for contrast (we build on provenance but enforce checks).
- **In-toto Supply Chain Framework:** in-toto’s documentation shows securing build steps via signed “layouts” and links【34†L209-L217】. Helps compare our trust-chain approach vs. traditional supply-chain attestation.
- **ZCAP-LD (Authorization Capabilities):** W3C Credentials Community Group materials on ZCAP-LD describe capability-based access control. Useful background on delegated authority (via JSON-LD contexts).
- **Runtime Verification:** The “Runtime Verification” lecture notes (Bollig et al., 2026) discuss monitoring under partial observability【26†L169-L177】. Highlights the concept of “inconclusive” monitors, motivating our consequence-based extension.
- **Active Learning:** The Nature Research summary on active learning【29†L60-L69】 explains query strategies (uncertainty, committee, information gain). Provides theory behind our observation queries.
- **Causal Inference / Counterfactuals:** Judea Pearl’s work on the do-calculus and causality (e.g. Pearl 2009) for context on interventions.
- **CEGAR (Abstraction Refinement):** Overview articles (Clarke et al., 2000; emergentmind summary【39†L40-L47】) describe iterative model checking, which inspired our counterexample-driven interventions.

These references inform novelty and claim drafting by highlighting differences between our proposed systems and known paradigms.
